RELATÓRIO DE AUDITORIA DE SEGURANÇA

Portal Institucional EEB Professor Benonívio João Martins

Data da Auditoria:	10 de Outubro de 2026
Ambiente & Versão:	Produção / v0.1.0 (Next.js 14.2.35, Supabase, Vercel)
Responsável Técnico:	Agente Especialista de Segurança (007 Hardening & SAST)
Status do Veredito:	**REPROVADO COM RESSALVAS CRÍTICAS** (Correções obrigatórias antes da homologação final)

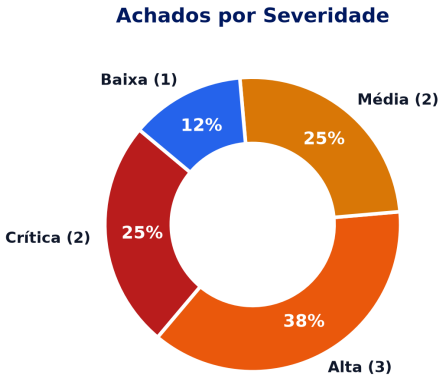
Nota Metodológica & Mapeamento de Stack

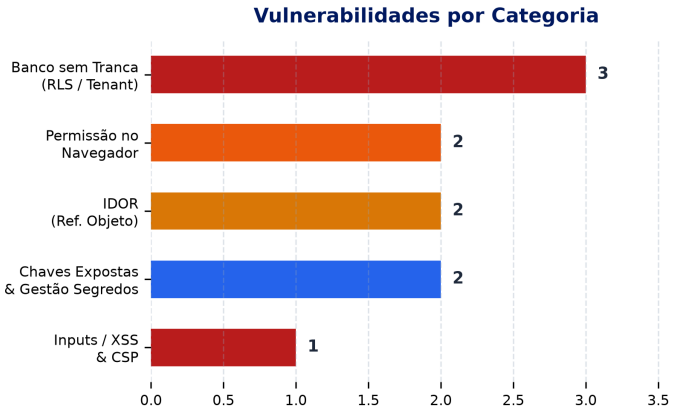
A auditoria cobriu 100% da base de código do portal escolar, abrangendo componentes de interface, Server Actions, rotas protegidas por middleware, esquemas relacionais no PostgreSQL e políticas de controle de acesso (Row Level Security - RLS). A análise adaptou estritamente as cinco categorias fundamentais para o ecossistema tecnológico detectado no projeto:

Stack Detectada	• **Linguagem:** TypeScript 5.x / Node.js 20+ • **Framework:** Next.js 14 (App Router, Server Actions, Server Components) • **Banco de Dados & ORM:** Supabase (PostgreSQL 17 gerenciado, RLS ativo) • **Autenticação:** Supabase Auth (JWT assinado via HMAC, cookies HttpOnly) • **Frontend:** React 18, Tailwind CSS, Lucide React • **Infra/Deploy:** Vercel Cloud (Git-driven). *Docker, Helm e Terraform ausentes.*
Mapeamento das 5 Categorias	1. **Banco sem Tranca:** Políticas RLS no Supabase PostgreSQL e filtros autoritativos de banco. 2. **Permissão no Navegador:** Gates no Next.js (Middleware vs. checagens nas Server Actions). 3. **IDOR:** Parâmetros de IDs em Server Actions manipulando tabelas sem validação de vínculo. 4. **Chaves Expostas:** Validação em runtime vs. build time e higiene de segredos no git/bundle. 5. **Inputs sem Tratamento:** Expressões regulares de sanitização, links de protocolo e CSP.

1. Resumo Executivo da Auditoria

Foram identificados **8 achados de segurança** relevantes, sendo **2 Críticos**, **3 Altos**, **2 Médios** e **1 Baixo**. Adicionalmente, **6 mecanismos defensivos robustos** foram verificados e atestados como implementados de forma exemplar.





CRÍTICA	ALTA	MÉDIA	BAIXA	PONTOS FORTES
2 Ação Imediata	**3** Correção P1	**2** Correção P2	**1** Melhoria P3	**6** Aprovados

Conclusão Geral: A arquitetura possui fundamentos de segurança avançados (Data Minimization, Rate Limiting, CSP estrito e sanitização criptográfica de uploads), mas sofre com vulnerabilidades lógicas de controle de acesso (RLS permissivo no Supabase, ausência de checagens de autorização em Server Actions de consulta e vetor de Stored XSS por bypass de protocolo).

2. Pontos Fortes e Riscos Centrais

[+] Pontos Fortes Atestados no Código Real:

- **Ausência Absoluta de Injeção HTML Bruta:** Varredura em 100% dos componentes React (arquivos .tsx) comprovou ausência total de dangerouslySetInnerHTML e innerHTML. O React escapa nativamente todas as strings de texto.
- **Parser Seguro de Hiperlinks:** O componente FormattedText (components/ui/formatted-text.tsx) valida estritamente URLs iniciando por http://, https:// ou www., eliminando execução de esquemas arbitrários em corpos de aviso.
- **Defesa Criptográfica contra Path Traversal:** Em upload.ts e storage-security.ts, nomes de arquivos são gerados compulsoriamente via crypto.randomUUID() e pastas higienizadas por regex, impossibilitando Directory Traversal.
- **Sliding-Window Rate Limiting em Camadas:** O módulo lib/rate-limit.ts impõe teto de 5 requisições/minuto no login (Edge Middleware e Server Action), com proteção anti-enumeração de contas (respostas de erro uniformes).
- **Sessões Seguras e Revogação Global:** O logout em login/actions.ts invoca supabase.auth.signOut({ scope: 'global' }), invalidando tokens em todos os dispositivos, e os cookies utilizam flags HttpOnly e Secure.
- **Higiene de Repositório e Git History Limpo:** Inspeção em todo o histórico de commits e no arquivo .gitignore confirmou inexistência de tokens privados ou arquivos .env commitados.

[!] Riscos Centrais Identificados:

- **Vazamento de Dados por RLS Aberto:** Políticas com FOR SELECT USING (true) sobrepõem cláusulas restritivas no PostgreSQL, permitindo que usuários anônimos baixem avisos confidenciais, rascunhos e registros na lixeira via PostgREST direto.
- **Bypass de Interface em Server Actions de Leitura:** 7 Server Actions administrativas não validam autenticação no servidor. Qualquer cliente pode invocá-las via POST com o cabeçalho Next-Action a partir de rotas públicas sem fazer login.
- **Vetor de Stored XSS por Regex Não-Recursivo:** lib/sanitize.ts remove javascript: apenas uma vez. Strings como javajavascript:script:alert(1) burlam a validação Zod e o filtro, sendo salvas no banco e executadas em links de turmas.
- **Ausência de RBAC:** Nenhuma Server Action checa se o usuário é Administrador; qualquer conta autenticada comum obtém poderes totais de mutação e deleção.
- **Validação de Segredos Apenas em Runtime:** lib/env-guard.ts não valida variáveis em tempo de build, permitindo que chaves prefixadas acidentalmente com NEXT_PUBLIC_ vazem para o bundle estático do cliente.

3. Tabela Detalhada de Achados por Categoria

A tabela a seguir apresenta arquivo por arquivo e linha por linha cada vulnerabilidade encontrada, sua criticidade e impacto prático.

SEV.	CATEGORIA	ARQUIVO:LINHA	DESCRIÇÃO DA VULNERABILIDADE & EXPLORABILIDADE
CRÍTICA	1. Banco sem Tranca	RLS_policies.sql:101-103 (banco real)	**RLS permissivo vazando rascunhos e lixeira:** A política USING (true) em escola_avisos combina com OR sobre a política de publicação, permitindo que visitantes anônimos leiam todos os avisos despublicados e na lixeira via API PostgREST.
CRÍTICA	5. Inputs / XSS	lib/sanitize.ts:18 turmas/actions.ts:48 turmas/[id]:93,174	**Stored XSS via bypass de protocolo javascript::** Remoção de padrão único em replace(/javascript:/gi, '') permite bypass com javajavascript:script:. O link malicioso é salvo e executado em <a href={link.url}>, agravado por CSP permissivo com unsafe-inline.
ALTA	2. Permissão no Navegador	acervo/actions.ts:45 avisos/actions.ts:50 turmas/actions.ts:82 + 4 arquivos	**7 Server Actions de leitura sem autenticação:** getAvisosAdmin, getAcervoAdmin, getDocumentosAdmin, etc., não invocam auth.getUser(). Qualquer usuário anônimo pode invocar o RPC direto via POST e extrair conteúdo confidencial.
ALTA	2. Permissão no Navegador	dashboard/*/actions.ts (mutações)	**Ausência de verificação RBAC (Role-Based Access):** Server Actions de escrita apenas testam se user existe, sem verificar se possui papel de admin. Qualquer conta autenticada tem poderes destrutivos totais.
ALTA	1. Banco sem Tranca	RLS_policies.sql:83-96 + todas as tabelas	**Inexistência de isolamento por proprietário:** Tabelas não possuem coluna user_id. RLS valida apenas auth.uid() IS NOT NULL, permitindo que qualquer usuário autenticado altere registros de outros.
MÉDIA	3. IDOR	turmas/actions.ts:313, 345, 388	**IDOR em recados de turmas:** Ações como deletePostPermanently recebem turma_id mas filtram apenas por id do post, permitindo manipulação cruzada entre diferentes turmas.
MÉDIA	4. Chaves Expostas	lib/env-guard.ts:6-24	**Guarda de segredos restrito a runtime:** validateApiIsolation() roda em tempo de execução. Variáveis NEXT_PUBLIC_ vazam para o bundle JavaScript estático durante o next build antes de qualquer checagem de runtime.
BAIXA	4. Chaves Expostas	lib/data/*.ts:4-7 (6 arquivos)	**Instanciação de cliente Supabase sem fail-fast:** Criação global com operador ! mascara ausência de variáveis de ambiente, causando crashes tardios sem validação na inicialização.

4. Plano de Remediação Priorizado

As recomendações a seguir foram estruturadas em ordem de impacto e esforço de implementação para orientar os desenvolvedores:

P1 (Imediata / Bloco de Produção)

- **Corrigir RLS no Supabase PostgreSQL:** Dropar as políticas permissivas abertas com USING (true) em escola_avisos, escola_acervo, escola_notificacoes e escola_turmas_posts. Manter leitura pública restrita a publicado = true AND status = 'ativo'.
- **Sanitização Recursiva e Validação Estrita de Protocolo:** Substituir a lógica de sanitizeString em lib/sanitize.ts por whitelist de protocolos: URLs devem iniciar compulsoriamente por http:// ou https://. Rejeitar terminantemente javascript: e data: via Zod regex.

P2 (Alta Prioridade / Próximo Deploy)

- **Blindar Server Actions de Leitura:** Adicionar const { data: { user } } = await supabase.auth.getUser(); if (!user) throw new Error('Não autorizado'); em getAvisosAdmin, getAcervoAdmin, getDocumentosAdmin, getNotificacoesAdmin, getPaginasAdmin, getTurmasAdmin e getPostsAdmin.
- **Implementar RBAC nas Server Actions:** Criar helper assertAdmin() que confere se user.app_metadata?.role === 'admin' antes de qualquer mutação de escrita ou deleção.
- **Corrigir Validação de Vínculo em Turmas (Anti-IDOR):** Em moveToTrashPost, restorePost e deletePostPermanently, adicionar cláusula .eq('turma_id', validTurmaId.data) na query de update/delete.

P3 (Média Prioridade / Hardening Contínuo)

- **Validação Pré-Build em next.config.mjs:** Inserir validateApiIsolation() no próprio next.config.mjs para abortar o processo de build caso segredos sejam inseridos em NEXT_PUBLIC_.
- **Endurecimento de CSP:** Remover 'unsafe-inline' e 'unsafe-eval' de script-src em next.config.mjs, adotando nonces ou hashes para scripts estáticos.

5. Issues para o GitHub (Prontas para Cópia)

Abaixo estão os textos completos formatados em Markdown para abertura direta de issues no GitHub, agrupados por escopo e contendo critérios de aceite verificáveis.

--- ISSUE 1 ---
Título: [Segurança] Vazamento de dados confidenciais e rascunhos por políticas RLS abertas (USING true)
Labels Sugeridas: security, severidade:crítica, rls, database
Descrição do Problema & Por Que é Explorável:
A política de leitura pública em `public.escola_avisos` e tabelas correlatas foi criada com `USING (true)`. No PostgreSQL RLS, políticas permissivas são unificadas via OR. Como resultado, qualquer usuário anônimo pode consultar diretamente a API PostgREST do Supabase (`/rest/v1/escola_avisos?select=*`) e visualizar avisos despublicados, rascunhos e itens na lixeira.
Evidência Técnica (Arquivo:Linha):

```
Arquivo: RLS_policies.sql:101-103
CREATE POLICY "Permitir leitura publica em avisos"
ON public.escola_avisos FOR SELECT
USING (true);
```

Impacto: Exposição pública de informações administrativas restritas, avisos futuros antes da publicação e itens descartados.
Sugestão de Correção:
Dropar a política `Permitir leitura publica em avisos` e manter exclusivamente uma política condicionada: `USING (publicado = true AND status = 'ativo')`. Replicar o endurecimento para `escola_acervo`, `escola_notificacoes` e `escola_turmas_posts`.
Critérios de Aceite (Checklist Verificável):
- [ ] Dropar política permissiva aberta no PostgreSQL Supabase
- [ ] Criar política de SELECT restrita a `publicado = true AND status = 'ativo'`
- [ ] Testar requisição curl anônima verificando que registros na lixeira retornam vazios
- [ ] Atualizar arquivo RLS_policies.sql no repositório
--- FIM ISSUE 1 ---

--- ISSUE 2 ---
Título: [Segurança] Vulnerabilidade de Stored XSS via esquema javascript: em links de turmas e recados
Labels Sugeridas: security, severidade:crítica, xss, frontend
Descrição do Problema & Por Que é Explorável:
O módulo `lib/sanitize.ts` utiliza regex não-recursivo `input.replace(/javascript:/gi, '')`. Uma carga como `javajavascript:script:alert(1)` burla a sanitização e passa na validação `z.string().url()`. O link é gravado no banco e renderizado diretamente em `<a href={link.url}>` na página pública de turmas (`app/(public)/turmas/[id]/page.tsx`). Agravado por CSP permissiva contendo `unsafe-inline`.
Evidência Técnica (Arquivo:Linha):

```
Arquivo: lib/sanitize.ts:18-19
.replace(/javascript:/gi, '')

Arquivo: app/(public)/turmas/[id]/page.tsx:93
<a href={link.url} target="_blank" rel="noopener noreferrer">
```

Impacto: Execução de código JavaScript arbitrário no navegador de visitantes, pais e estudantes ao clicar em links da turma.
Sugestão de Correção:
1. Modificar o schema Zod para rejeitar qualquer protocolo diferente de `http:` ou `https:`:
`z.string().regex(/^https?:\/\//i, 'O link deve iniciar com http:// ou https://')`
2. Remover `unsafe-inline` da Content-Security-Policy em `next.config.mjs`.
Critérios de Aceite (Checklist Verificável):
- [ ] Validar protocolo estrito `http://` e `https://` no Zod schema de links
- [ ] Bloquear terminantemente esquemas `javascript:` e `data:`
- [ ] Implementar helper de sanitização de URLs antes da renderização de href
- [ ] Validar bloqueio contra payloads aninhados no teste unitário
--- FIM ISSUE 2 ---

--- ISSUE 3 ---
Título: [Segurança] Server Actions administrativas sem autenticação no servidor (Bypass de Middleware)
Labels Sugeridas: security, severidade:alta, auth, server-actions
Descrição do Problema & Por Que é Explorável:
Sete Server Actions de listagem administrativa (`getAvisosAdmin`, `getAcervoAdmin`, `getDocumentosAdmin`, `getNotificacoesAdmin`, `getPaginasAdmin`, `getTurmasAdmin` e `getPostsAdmin`) não validam a sessão com `supabase.auth.getUser()`. Qualquer invasor pode disparar um POST direto com cabeçalho `Next-Action` a partir de páginas públicas sem passar pelo middleware.
Evidência Técnica (Arquivo:Linha):

```
Arquivo: app/(admin)/dashboard/avisos/actions.ts:50-58
export async function getAvisosAdmin() {
const supabase = createClient();
const { data, error } = await supabase.from('escola_avisos').select(...);
```

Impacto: Extração irrestrita de dados do painel de controle por agentes externos não autenticados.
Sugestão de Correção:
Inserir em todas as Server Actions de consulta administrativa:
```ts
const supabase = createClient();
const { data: { user }, error } = await supabase.auth.getUser();
if (error || !user) throw new Error('Não autorizado');
```
Critérios de Aceite (Checklist Verificável):
- [ ] Adicionar `auth.getUser()` em `getAvisosAdmin`
- [ ] Adicionar `auth.getUser()` em `getAcervoAdmin`
- [ ] Adicionar `auth.getUser()` em `getDocumentosAdmin`
- [ ] Adicionar `auth.getUser()` em `getNotificacoesAdmin`
- [ ] Adicionar `auth.getUser()` em `getPaginasAdmin`
- [ ] Adicionar `auth.getUser()` em `getTurmasAdmin` e `getPostsAdmin`
- [ ] Verificar que chamadas anônimas via POST retornam erro de autorização
--- FIM ISSUE 3 ---

--- ISSUE 4 ---
Título: [Segurança] Falha de IDOR e inconsistência de escopo em posts de turmas
Labels Sugeridas: security, severidade:média, idor, turmas
Descrição do Problema & Por Que é Explorável:
As Server Actions de exclusão e restauração de posts de turmas (`deletePostPermanently`, `moveToTrashPost`, `restorePost`) recebem o ID da turma como parâmetro, mas executam a mutação filtrando unicamente pelo `id` do post, permitindo alterar ou apagar comunicados pertencentes a turmas diferentes.
Evidência Técnica (Arquivo:Linha):

```
Arquivo: app/(admin)/dashboard/turmas/actions.ts:388-392
const { error: deleteError } = await supabase
.from('escola_turmas_posts')
.delete()
.eq('id', validId.data);
```

Impacto: Possibilidade de corrupção ou exclusão de comunicados de turmas de terceiros através de IDs manipulados.
Sugestão de Correção:
Incluir na query a condição de vínculo hierárquico:
`.eq('id', validId.data).eq('turma_id', validTurmaId.data)`
Critérios de Aceite (Checklist Verificável):
- [ ] Adicionar filtro `.eq('turma_id', validTurmaId.data)` em `moveToTrashPost`
- [ ] Adicionar filtro `.eq('turma_id', validTurmaId.data)` em `restorePost`
- [ ] Adicionar filtro `.eq('turma_id', validTurmaId.data)` em `deletePostPermanently`
- [ ] Validar retorno de erro caso o post não pertença à turma especificada
--- FIM ISSUE 4 ---

--- ISSUE 5 ---
Título: [Segurança] Validação de isolamento de segredos em tempo de compilação (Build-Time Guard)
Labels Sugeridas: security, severidade:média, env, build
Descrição do Problema & Por Que é Explorável:
A rotina `validateApiIsolation()` em `lib/env-guard.ts` opera somente em tempo de execução. Como o Next.js embute variáveis com prefixo `NEXT_PUBLIC_` estaticamente durante o comando `next build`, chaves confidenciais configuradas incorretamente vazam para o bundle JavaScript do navegador antes do runtime.
Evidência Técnica (Arquivo:Linha):

```
Arquivo: lib/env-guard.ts:6-24
export function validateApiIsolation(): void { ... }
```

Impacto: Exposição permanente de chaves de API caso ocorra erro humano na parametrização de variáveis na Vercel.
Sugestão de Correção:
Invocar `validateApiIsolation()` no topo de `next.config.mjs` para interromper o build com código de erro caso qualquer segredo seja detectado em variáveis com prefixo público.
Critérios de Aceite (Checklist Verificável):
- [ ] Importar ou reproduzir `validateApiIsolation()` em `next.config.mjs`
- [ ] Testar falha de build forçando variável inválida `NEXT_PUBLIC_TEST_KEY=AIzaSy...`
- [ ] Garantir que o pipeline da Vercel aborte antes de gerar artefatos estáticos
--- FIM ISSUE 5 ---